

14 — Dynamic Programming (2D)

DSA in Python — Interview Prep

Dynamic Programming (2D) — Tutorial

Covers the 10 problems solved in this folder. Same underlying idea as [13_dynamic_programming_1d](#) — build up from smaller subproblems — but the state now needs *two* indices to describe: a position in each of two strings, a row and column in a grid, or an item index paired with a remaining capacity. Several of the "hardest" problems here (06, 07) don't look like grid/knapsack problems at all from their statements — recognizing the disguise is most of the difficulty.

The patterns

1. Grid DP — position determined by (row, col) (01, 02)

The most literal 2D DP: `dp[r][c]` describes something about reaching cell (r, c) , built from the cells directly above and to the left (the only ways in, given the allowed moves). `unique_paths` sums the two predecessors (count of ways); `minimum_path_sum` takes their `min` plus the current cell's own cost. Same shape, different combining operation — the same lesson as 1D DP's rolling-pair problems, just one dimension up.

2. Two-string DP — comparing prefixes of two sequences (03, 04, 10)

`dp[i][j]` describes a relationship between the first i characters of one string and the first j characters of another. The recurrence almost always branches on whether the *current* characters match:

- `longest_common_subsequence`: match $\rightarrow$ extend diagonally (`dp[i-1][j-1] + 1`); mismatch $\rightarrow$ best of dropping one side (`max(dp[i-1][j], dp[i][j-1])`).
- `edit_distance`: match $\rightarrow$ carry forward unchanged (`dp[i-1][j-1]`); mismatch $\rightarrow 1 + \min$ of three specific edits, each mapping to a specific neighbor cell (replace = diagonal, delete = up, insert = left). Structurally LCS with `max` replaced by `min` and an unconditional `+1` added for the mismatch case.
- `interleaving_string`: a variant where i and j don't get compared to *each other* — they jointly point at a fixed position in a *third* string, and the question is whether either side's next character can extend a valid interleaving.

3. 0/1 Knapsack and its disguises (05, 06, 07)

`zero_one_knapsack` is the base case: for each item, skip it (`dp[i-1][cap]` unchanged) or take it (`dp[i-1][cap-weight] + value`, only if it fits) — critically reading from row $i-1$ (not row i), since each item can only be used once. This is what distinguishes it from [13_dynamic_programming_1d](#)'s coin-change problems, where items (coins) can repeat and the lookup stays on the same row/pass.

- `partition_equal_subset_sum`: not phrased as a knapsack at all, but "does some subset sum to $\text{total}/2$ " is exactly "is capacity $\text{total}/2$ exactly fillable" with each item's value equal to its weight — a boolean reachability version of the knapsack table.

- **target_sum**: an algebra trick ($\text{sum}(P) - \text{sum}(N) = \text{target}$, $\text{sum}(P) + \text{sum}(N) = \text{total}$, solved together) reduces "assign +/- signs" down to "count subsets summing to a fixed target" — the *counting* version of 06's feasibility check.

Recognizing "this is secretly 0/1 knapsack" is the single most valuable pattern-matching skill this folder builds — neither 06 nor 07 mentions weights, values, or capacity anywhere in its problem statement.

4. Palindrome table filled by increasing length (08, 09)

$\text{dp}[i][j]$ = "is $s[i..j]$ a palindrome?" True when the outer characters match *and* the strictly-inside substring is also a palindrome ($\text{dp}[i+1][j-1]$). The fill order matters here in a way it didn't for grid DP: the table must be filled by increasing *substring length*, not row-by-row or column-by-column, because computing a length-L answer needs a strictly shorter substring's answer to already be finalized. Once this table exists, different final questions read off of it almost for free: 08 tracks the best (longest) True cell seen; 09 just counts every True cell — same table, two different one-line readouts.

Complexity summary

#	Problem	Time	Extra space
01	unique_paths	$O(m \cdot n)$	$O(m \cdot n)$
02	minimum_path_sum	$O(m \cdot n)$	$O(m \cdot n)$
03	longest_common_subsequence	$O(m \cdot n)$	$O(m \cdot n)$
04	edit_distance	$O(m \cdot n)$	$O(m \cdot n)$
05	zero_one_knapsack	$O(n \cdot \text{capacity})$	$O(n \cdot \text{capacity})$
06	partition_equal_subset_sum	$O(n \cdot \text{target})$	$O(\text{target})$
07	target_sum	$O(n \cdot \text{subset_sum})$	$O(\text{subset_sum})$
08	longest_palindromic_substring	$O(n^2)$	$O(n^2)$
09	palindromic_substrings	$O(n^2)$	$O(n^2)$
10	interleaving_string	$O(m \cdot n)$	$O(m \cdot n)$

06 and 07 show 1D space (not 2D) despite being knapsack-shaped — because each item is only used once, the capacity dimension can be collapsed to a single rolling array *if* it's iterated backward per item (same reasoning as 13_dynamic_programming_1d/06_coin_change_ii's loop-order requirement, adapted to the no-reuse setting).

Common interview follow-ups to be ready for

- **unique_paths / minimum_path_sum**: "what if some cells are blocked/obstacles?" — set $\text{dp}[r][c] = 0$ (unreachable) or skip that cell in the min entirely for blocked cells; the rest of the recurrence is unchanged.
- **longest_common_subsequence**: "can you reconstruct the actual subsequence, not just its length?" — walk backward from $\text{dp}[m][n]$: on a match go diagonally and record the character; on a mismatch, step toward whichever neighbor the max came from.
- **edit_distance**: "can you print the actual sequence of edits?" — same backward-walk idea as LCS, tracking which of the three options (replace/delete/insert) produced each cell's value.

- `partition_equal_subset_sum`: "what if you needed exactly k subsets of equal sum, not just 2?" — a genuinely harder problem (partition into k equal-sum subsets), usually solved with backtracking + pruning rather than a direct DP extension; good to know this doesn't trivially generalize.
- `target_sum`: be ready to derive the $\text{sum}(P) = (\text{target} + \text{total}) / 2$ algebra live, not just state the formula — that derivation is usually exactly what's being tested.
- `longest_palindromic_substring`: "can you do this in $O(n)$ instead of $O(n^2)$?" — yes, Manacher's algorithm; know it exists as the advanced follow-up, but the $O(n^2)$ DP here is the expected first-derived solution.
- `interleaving_string`: "what if there were three source strings interleaving into a fourth?" — the 2D table would need to become 3D (one dimension per source string), same core recurrence idea extended by one more dimension.

Self-check before moving on

You should be able to, without looking at the code:

- Write the grid-DP recurrence for both `unique_paths` and `minimum_path_sum` from memory and state what changes between them.
- Explain the LCS match/mismatch recurrence, then adapt it live into edit distance's three-way min.
- Explain why `zero_one_knapsack` reads from row $i-1$ while `13_dynamic_programming_1d/05_coin_change` reads from the same row/pass, and connect that difference to "each item used once" vs. "unlimited reuse."
- Derive `target_sum`'s reduction to subset-sum counting on paper, starting from the two equations.
- Explain why the palindrome table in `08/09` must be filled by increasing substring length rather than row-by-row.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `15_greedy`.

Programs

01. Unique Paths

01_unique_paths.py

Explanation

Problem: A robot starts at the top-left of an $m \times n$ grid and can only move right or down. Return the number of distinct paths to the bottom-right corner.

Source : LeetCode 62 - Unique Paths

Example:

Input: $m = 3, n = 7$

Output: 28

Idea: the simplest possible 2D DP - $dp[r][c]$ = number of ways to reach cell (r, c) . Every cell can only be entered from directly above or directly left (the only two allowed moves), so the number of ways to reach it is simply the sum of the ways to reach those two neighbors: $dp[r][c] = dp[r-1][c] + dp[r][c-1]$. The entire first row and first column are seeded with 1 (there's exactly one way to reach any cell along an edge - keep moving in the only direction available) rather than needing a special case in the main loop.

Code

```
def unique_paths(m: int, n: int) -> int:
    dp = [[1] * n for _ in range(m)]

    for r in range(1, m):
        for c in range(1, n):
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]

    return dp[m - 1][n - 1]

if __name__ == "__main__":
    tests = [
        (3, 7, 28),
        (3, 2, 3),
        (1, 1, 1),
        (7, 3, 28),
    ]

    for i, (m, n, expected) in enumerate(tests, 1):
        got = unique_paths(m, n)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Minimum Path Sum

02_minimum_path_sum.py

Explanation

Problem: Given a grid of non-negative numbers, find a path from top-left to bottom-right (only moving right or down) that minimizes the sum of numbers along the path.

Source : LeetCode 64 - Minimum Path Sum

Example:

Input: `[[1,3,1],[1,5,1],[4,2,1]]`

Output: 7 (path 1 -> 3 -> 1 -> 1 -> 1)

Idea: same shape as 01, `min` instead of `+` for combining the two possible predecessors, and this time the **cost** of each cell (not just a count of 1 way per step) has to be folded in. `dp[r][c]` = the grid's own cost at (r, c), plus whichever predecessor (above or left) had the cheaper path to reach it. The first row and column are seeded by accumulating along the only available direction, same reasoning as 01's "only one way in along an edge," just summing costs instead of counting paths.

Code

```
def min_path_sum(grid: list[list[int]]) -> int:
    rows, cols = len(grid), len(grid[0])
    dp = [[0] * cols for _ in range(rows)]

    dp[0][0] = grid[0][0]
    for c in range(1, cols):
        dp[0][c] = dp[0][c - 1] + grid[0][c]
    for r in range(1, rows):
        dp[r][0] = dp[r - 1][0] + grid[r][0]

    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])

    return dp[rows - 1][cols - 1]

if __name__ == "__main__":
    tests = [
        ([[1, 3, 1], [1, 5, 1], [4, 2, 1]], 7),
        ([[1, 2, 3], [4, 5, 6]], 12),
        ([[5]], 5),
    ]

    for i, (grid, expected) in enumerate(tests, 1):
        got = min_path_sum(grid)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Longest Common Subsequence

03_longest_common_subsequence.py

Explanation

Problem: Given two strings text1 and text2, return the length of their longest common subsequence (characters in the same relative order in both, not necessarily contiguous). Return 0 if none.

Source : LeetCode 1143 - Longest Common Subsequence

Example:

Input: text1 = "abcde", text2 = "ace"

Output: 3 ("ace" is a subsequence of both)

Idea: the canonical *two-string* 2D DP - $dp[i][j]$ = the LCS length using the first i characters of text1 and the first j characters of text2. Two cases at each cell: if the current characters match ($text1[i-1] == text2[j-1]$), they can both be included in the subsequence, so extend the LCS found *without* either of them ($dp[i-1][j-1] + 1$). If they don't match, the best is whichever is better - dropping the current character of text1 or dropping the current character of text2 ($\max(dp[i-1][j], dp[i][j-1])$). This "match -> extend diagonally; mismatch -> take the best of dropping one side" shape is the foundation 04's edit distance builds on directly.

Code

```
def longest_common_subsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

if __name__ == "__main__":
    tests = [
        ("abcde", "ace", 3),
        ("abc", "abc", 3),
        ("abc", "def", 0),
        ("", "abc", 0),
    ]

    for i, (text1, text2, expected) in enumerate(tests, 1):
        got = longest_common_subsequence(text1, text2)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Edit Distance

04_edit_distance.py

Explanation

Problem: Given two words, return the minimum number of single-character operations (insert, delete, or replace) needed to turn word1 into word2.

Source : LeetCode 72 - Edit Distance (Levenshtein Distance)

Example:

Input: word1 = "horse", word2 = "ros"

Output: 3 (horse -> rorse -> rose -> ros)

Idea: same $dp[i][j]$ -over-two-strings shape as O3, but instead of "longest match," this counts "fewest edits." If the current characters match, no edit is needed here - carry forward $dp[i-1][j-1]$ unchanged (mirrors O3's diagonal-match case exactly). If they don't match, one of three single edits must happen, and each maps to a specific neighbor in the table: replace word1[i-1] with word2[j-1] ($dp[i-1][j-1] + 1$ - both strings advance together after the edit), delete word1[i-1] ($dp[i-1][j] + 1$ - word1 advances alone), or insert word2[j-1] into word1 ($dp[i][j-1] + 1$ - word2 advances alone). Taking the minimum of those three options is what makes this "edit distance" where O3 took a **max** for "longest match" - same table shape, inverted optimization goal. The first row/column are seeded with i and j (turning an i-length prefix into empty, or empty into a j-length prefix, costs exactly that many inserts/deletes).

Code

```
def min_distance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j - 1], # replace
                    dp[i - 1][j],     # delete from word1
                    dp[i][j - 1],     # insert into word1
                )

    return dp[m][n]

if __name__ == "__main__":
    tests = [
        ("horse", "ros", 3),
        ("intention", "execution", 5),
        ("", "abc", 3),
        ("abc", "abc", 0),
    ]

    for i, (word1, word2, expected) in enumerate(tests, 1):
```

```

got = min_distance(word1, word2)
status = "PASS" if got == expected else "FAIL"
print(f"Test {i}: {status} (got {got}, expected {expected})")

```

05. Zero One Knapsack

05_zero_one_knapsack.py

Explanation

Problem: Given item weights, item values, and a knapsack capacity, choose a subset of items (each item used at most once - "0/1" meaning include-or-don't, no fractions or repeats) that maximizes total value without exceeding capacity.

Source : classic 0/1 Knapsack (foundational DP problem, underlies 06 and 07 in this folder)

Example:

Input: weights = [1, 3, 4, 5], values = [1, 4, 5, 7], capacity = 7

Output: 9 (take items of weight 3 and 4, value 4 + 5)

Idea: $dp[i][cap]$ = the best value achievable using only the first i items, with capacity cap . For each item, there are exactly two choices: skip it (value = $dp[i-1][cap]$, unchanged) or take it (only possible if it fits: value = $dp[i-1][cap - weight] + value$, since taking it uses up **weight** capacity and this item can't be taken again - the "0/1" constraint is exactly why the lookup goes to row $i-1$, not row i , unlike [13_dynamic_programming_1d/05_coin_change](#)'s unbounded version, which reuses row i because coins can repeat). $dp[i][cap]$ is the better of those two choices. This "skip or take, constrained by remaining capacity" table is the base every 0/1-knapsack-*shaped* problem (06, 07) reduces down to, even when the problem statement doesn't mention weights/values/capacity at all.

Code

```

def knapsack(weights: list[int], values: list[int], capacity: int) -> int:
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        weight, value = weights[i - 1], values[i - 1]
        for cap in range(capacity + 1):
            dp[i][cap] = dp[i - 1][cap]
            if weight <= cap:
                dp[i][cap] = max(dp[i][cap], dp[i - 1][cap - weight] + value)

    return dp[n][capacity]

if __name__ == "__main__":
    tests = [
        ([1, 3, 4, 5], [1, 4, 5, 7], 7, 9),
        ([2, 3, 4, 5], [3, 4, 5, 6], 5, 7),
        ([10], [100], 5, 0),
        ([], [], 10, 0),
    ]

```

```

]

for i, (weights, values, capacity, expected) in enumerate(tests, 1):
    got = knapsack(weights, values, capacity)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

06. Partition Equal Subset Sum

06_partition_equal_subset_sum.py

Explanation

Problem: Given an array of positive integers, determine if it can be partitioned into two subsets with equal sum.

Source : LeetCode 416 - Partition Equal Subset Sum

Example:

Input: [1, 5, 11, 5]

Output: True ([1, 5, 5] and [11] both sum to 11)

Idea: not obviously a knapsack problem from the statement, but it is one: if the total sum is odd, an equal split is impossible immediately (no need to search at all). Otherwise, the question becomes "does some subset sum to exactly total/2?" - which is 05's knapsack with every item's *value* equal to its *weight* and capacity set to target: "can capacity be filled exactly" instead of "what's the best value that fits." Each number is an item, taken (included in the subset) or skipped, same as 05 - just reframed as a boolean reachability table (`dp[s]` = "is sum *s* achievable?") instead of a value-maximization table. The space is rolled down to 1D here (unlike 05's explicit 2D table) by iterating the capacity dimension *backward* for each item - the same "don't let one item update a value that the same item might read again later in the same pass" concern as [13_dynamic_programming_1d/06_coin_change_ii](#)'s loop-order requirement, adapted to the 0/1 (no-reuse) setting.

Code

```

def can_partition(nums: list[int]) -> bool:
    total = sum(nums)
    if total % 2 != 0:
        return False

    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True

    for n in nums:
        for s in range(target, n - 1, -1):
            if dp[s - n]:
                dp[s] = True

    return dp[target]

if __name__ == "__main__":

```

```

tests = [
    ([1, 5, 11, 5], True),
    ([1, 2, 3, 5], False),
    ([1, 1], True),
    ([1], False),
]

for i, (nums, expected) in enumerate(tests, 1):
    got = can_partition(nums)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

07. Target Sum

07_target_sum.py

Explanation

Problem: Given an array of non-negative integers and a target, assign a '+' or '-' sign to each number so the resulting expression equals target. Return the number of ways to do so.

Source : LeetCode 494 - Target Sum

Example:

Input: nums = [1, 1, 1, 1, 1], target = 3

Output: 5

Idea: not obviously a knapsack problem either, but a small algebra trick reveals it is one. Split nums into a "positive" subset P and a "negative" subset N (every number goes to exactly one side, based on its assigned sign). Then $\text{sum}(P) - \text{sum}(N) = \text{target}$, and also $\text{sum}(P) + \text{sum}(N) = \text{total}$ (the sum of all numbers, sign-free). Adding those two equations: $2 \cdot \text{sum}(P) = \text{target} + \text{total}$, so $\text{sum}(P) = (\text{target} + \text{total}) / 2$ - a fixed, computable number. The problem has been reduced to "how many subsets of nums sum to exactly $\text{sum}(P)$ " - the *counting* version of 06's feasibility knapsack, counting ways instead of just checking reachability ($\text{dp}[s] += \text{dp}[s-n]$, same shape as [13_dynamic_programming_1d/06_coin_change_ii](#), but 0/1 instead of unbounded, hence the same backward-iteration requirement as 06). The upfront parity/bounds check $((\text{total} + \text{target}) \% 2 \neq 0 \text{ or } \text{total} < \text{abs}(\text{target}))$ catches target values no sign assignment could ever reach, before running the DP at all.

Code

```

def find_target_sum_ways(nums: list[int], target: int) -> int:
    total = sum(nums)
    if (total + target) % 2 != 0 or total < abs(target):
        return 0

    subset_sum = (total + target) // 2
    dp = [0] * (subset_sum + 1)
    dp[0] = 1

```

```

    for n in nums:
        for s in range(subset_sum, n - 1, -1):
            dp[s] += dp[s - n]

    return dp[subset_sum]

if __name__ == "__main__":
    tests = [
        ([1, 1, 1, 1, 1], 3, 5),
        ([1], 1, 1),
        ([1], 2, 0),
        ([1, 0], 1, 2),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = find_target_sum_ways(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Longest Palindromic Substring

08_longest_palindromic_substring.py

Explanation

Problem: Given a string *s*, return its longest palindromic substring (contiguous, unlike a subsequence).

Source : LeetCode 5 - Longest Palindromic Substring

Example:

Input: "babad"

Output: "bab" ("aba" is also a valid answer)

Idea: $dp[i][j]$ = "is $s[i..j]$ a palindrome?" A substring is a palindrome exactly when its outer characters match ($s[i] == s[j]$) *and* everything strictly inside is also a palindrome ($dp[i+1][j-1]$) - or the inside is trivially empty/a single character (length 2 or less), which needs no further check. Filling the table by increasing *length* (not by row or column directly) is what makes this correct: computing $dp[i][j]$ for a substring of length *L* requires $dp[i+1][j-1]$, a strictly shorter substring, which must already be finalized - iterating outer-to-inner lengths guarantees that dependency is always ready before it's needed. Track the best (start, length) seen as the table fills.

Code

```

def longest_palindromic_substring(s: str) -> str:
    n = len(s)
    if n == 0:
        return ""

    dp = [[False] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = True

```

```

start, max_len = 0, 1
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j] and (length == 2 or dp[i + 1][j - 1]):
            dp[i][j] = True
            if length > max_len:
                start, max_len = i, length

return s[start:start + max_len]

if __name__ == "__main__":
    tests = [
        ("babad", {"bab", "aba"}),
        ("cbbd", {"bb"}),
        ("a", {"a"}),
        ("", {""}),
    ]

    for i, (s, expected_set) in enumerate(tests, 1):
        got = longest_palindromic_substring(s)
        status = "PASS" if got in expected_set else "FAIL"
        print(f"Test {i}: {status} (got {got!r}, expected one of {expected_set})")

```

09. Palindromic Substrings

09_palindromic_substrings.py

Explanation

Problem: Given a string *s*, return the total number of palindromic substrings it contains (different positions count separately, even if the substrings have the same characters).

Source : LeetCode 647 - Palindromic Substrings

Example:

Input: "aaa"

Output: 6 ("a", "a", "a", "aa", "aa", "aaa")

Idea: the exact same `dp[i][j]` palindrome table as 08 - the only difference is what's done with it. Instead of tracking the single best (longest) palindrome found, every cell that's True is itself a valid palindromic substring, so simply counting how many cells end up True gives the answer directly. This pairing (08 and 09) is worth noticing as a general lesson: once a DP table answers "is X true for this state," many different final questions (longest, count, does any exist, list them all) can be read off the **same** table - the table itself is usually the hard part; the final readout is often trivial.

Code

```

def count_substrings(s: str) -> int:
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    count = 0

    for length in range(1, n + 1):

```

```

        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and (length <= 2 or dp[i + 1][j - 1]):
                dp[i][j] = True
                count += 1

    return count

if __name__ == "__main__":
    tests = [
        ("abc", 3),
        ("aaa", 6),
        ("", 0),
        ("aba", 4),
    ]

    for i, (s, expected) in enumerate(tests, 1):
        got = count_substrings(s)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

10. Interleaving String

10_interleaving_string.py

Explanation

Problem: Given strings s1, s2, and s3, determine if s3 is formed by interleaving s1 and s2 (s3 uses every character of both, preserving each string's internal relative order, but the two can be mixed together in any order).

Source : LeetCode 97 - Interleaving String

Example:

Input: s1 = "aabcc", s2 = "dbbca", s3 = "aadbcbcbac"

Output: True

Idea: back to a two-string dp[i][j] table like 03/04, but here both i and j jointly determine a *fixed* position in s3 (s3[i+j-1]) rather than being compared to each other directly - the length constraint `len(s1) + len(s2) == len(s3)` must hold first, or interleaving is impossible regardless of content. dp[i][j] = "can the first i characters of s1 and first j characters of s2 interleave to form the first i+j characters of s3?" True exactly when *either* s1's next character extends a valid interleaving from dp[i-1][j], or s2's next character extends one from dp[i][j-1] - matching the appropriate side against s3[i+j-1] in each case. The first row and column are seeded by checking whether s1 alone (or s2 alone) matches the corresponding prefix of s3 exactly, since only one string is contributing along those edges.

Code

```

def is_interleave(s1: str, s2: str, s3: str) -> bool:
    m, n = len(s1), len(s2)

```

```

if m + n != len(s3):
    return False

dp = [[False] * (n + 1) for _ in range(m + 1)]
dp[0][0] = True

for i in range(1, m + 1):
    dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]
for j in range(1, n + 1):
    dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

for i in range(1, m + 1):
    for j in range(1, n + 1):
        dp[i][j] = (
            (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1])
            or (dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])
        )

return dp[m][n]

if __name__ == "__main__":
    tests = [
        ("aabcc", "dbbca", "aadbcbcbac", True),
        ("aabcc", "dbbca", "aadbcbaccc", False),
        ("", "", "", True),
        ("a", "", "a", True),
        ("a", "b", "ab", True),
    ]

    for i, (s1, s2, s3, expected) in enumerate(tests, 1):
        got = is_interleave(s1, s2, s3)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```